INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

# SISTEMA MULTIEMPRESA DE GESTIÓN VETERINARIA

## MANUAL TÉCNICO · VERSIÓN FINAL

itq

LUIS ALMACHI · EMILIO LÓPEZ · CRISTOPHER SANTANDER
SEBASTIÁN GONZAGA · ÁNGEL GALARZA

**TUTOR: M.Sc. ELVIS PACHACAMA**

INSTITUTO SUPERIOR TECNOLÓGICO QUITO

# MANUAL TÉCNICO FINAL

*Instalación local y despliegue público · versión 2.0*

Proyecto: ControlPets / Sistema multiempresa de gestión veterinaria | Revisión documental: 05 de noviembre de 2026

Institución: Instituto Superior Tecnológico Quito (ITQ). Documento académico de referencia para revisión y mantenimiento del proyecto.

## Guía de contenidos

01 Alcance e infraestructura

02 Estructura tecnológica

03 Base PostgreSQL y respaldos

04 Configuración de variables

05 Despliegue local

06 Docker y Render

07 Vercel

08 Seguridad y endpoints

09 Actualizaciones y migraciones

10 Diagnóstico y respaldo

Los números de sección facilitan la consulta. La paginación puede variar al editar esta versión Word.

## 1. Alcance e infraestructura

ControlPets es un sistema SaaS multiempresa. La interfaz React se aloja en Vercel; el backend Java compilado como WAR se ejecuta en Tomcat sobre Render; PostgreSQL 18 aloja la información centralizada en otra instancia de Render.

| Capa | Tecnología | Producción (verificada) |
|---|---|---|
| Frontend | React + TypeScript + Vite | https://controlpets.vercel.app |
| Backend | Java 17 / Jakarta Servlet 6 / Tomcat 10.1 | https://controlpets-api.onrender.com/VETERINARIAITQ/ |
| Persistencia | PostgreSQL 18, JDBC, HikariCP | Base Render controlpets |
| Build y versión | Maven, npm, Git / Docker | gonzagagzg/ControlPets, rama Desarrollo |

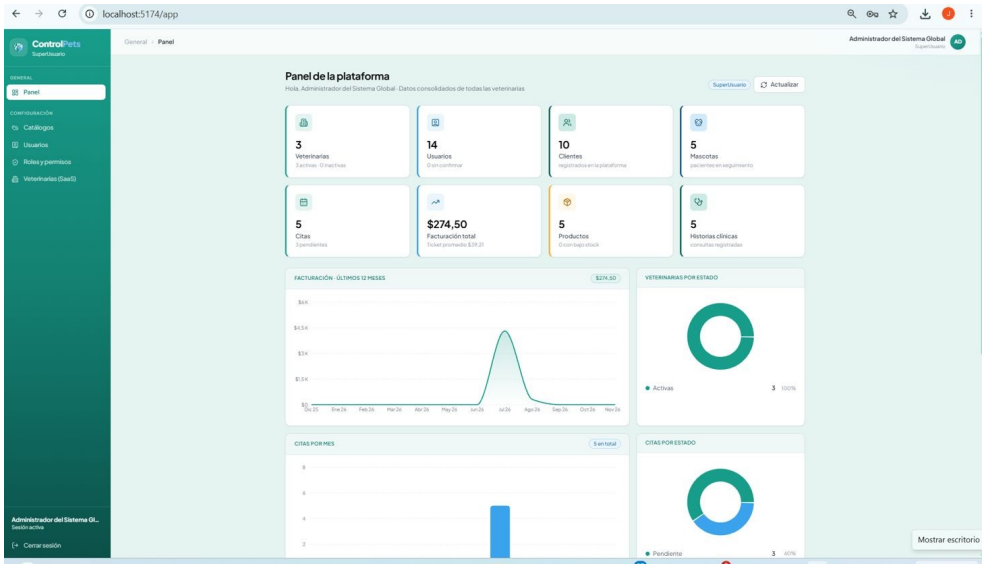



Figura 1. ControlPets en ejecución: panel global y datos de ejemplo.

## 2. Estructura tecnológica

```
ControlPets/
  Backend/  pom.xml · src/main/java/com/itq/{config,filter,servlet,service,dao,...}
            src/main/resources/sql/ · Dockerfile · .dockerignore
  Frontend/ package.json · src/app/router.tsx · src/features/ · src/shared/api/
            vercel.json · .env (local, no publicar)
  BD/       scripts y respaldo histórico
  Test_documentacion/ manuales e informes institucionales
```

El flujo de petición es: navegador HTTPS → API REST/Servlets → filtros JWT y CORS → servicios → DAO/HikariCP → PostgreSQL. La API responde generalmente con el sobre JSON {exito, mensaje, datos}.

## 3. Base de datos y respaldos

La base utilizada en la revisión fue aplicacion_veterinaria (local, PostgreSQL 18) y controlpets (Render, PostgreSQL 18). La importación verificada registró 24 tablas, y las credenciales del usuario de aplicación deben coincidir con las configuradas en cada entorno.

```
SELECT current_database();
SELECT COUNT(*) AS total_tablas
FROM information_schema.tables
WHERE table_schema = 'public' AND table_type = 'BASE TABLE';
```

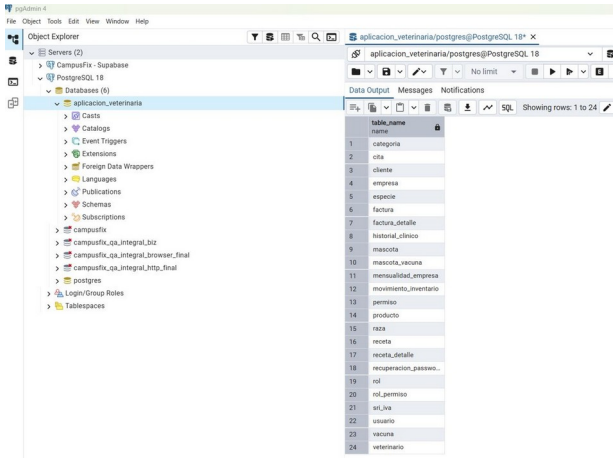



Figura 2. Verificación real de 24 tablas en PostgreSQL local.

### 3.1. Respaldo y restauración

```
pg_dump -h localhost -p 5432 -U postgres -d aplicacion_veterinaria -Fc -f ControlPets_actual.backup
pg_restore --no-owner --no-acl --exit-on-error -d "$URL_RENDER" ControlPets_actual.backup
```

Un respaldo pg_dump -Fc se restaura con pg_restore, no en Query Tool. En Render, restaurar solo en una base vacía; no repetir el respaldo sobre tablas que ya contienen datos. Conservar respaldo previo a cada migración.

## 4. Variables de entorno: backend y frontend

Cada integrante configura Backend/.env con su PostgreSQL local. Render mantiene variables propias en Environment; estas no deben guardarse en GitHub. No reproducir claves reales en manuales.

```
VET_DB_HOST=localhost
VET_DB_PORT=5432
VET_DB_NAME=aplicacion_veterinaria
VET_DB_USER=postgres
VET_DB_PASSWORD=<CLAVE_LOCAL>
JWT_SECRET=<CLAVE_ALEATORIA_SEGURA>
FRONTEND_URL=http://localhost:5174
CORS_ORIGIN=http://localhost:5174
SERVER_PORT=8080
SERVER_CONTEXT_PATH=/VETERINARIAITQ
```

En producción, VET_DB_HOST es el hostname interno de Render y VET_DB_NAME es controlpets. Añada las variables SMTP únicamente si utiliza las funciones de correo; Render Free puede restringir tráfico SMTP saliente.

```
Frontend/.env (local)
VITE_API_URL=http://localhost:8080/VETERINARIAITQ

Vercel / Production:
VITE_API_URL=https://controlpets-api.onrender.com/VETERINARIAITQ
```

La URL del frontend no debe terminar en /api porque el cliente añade las rutas /api/... en las llamadas.

## 5. Instalación y ejecución local

1. Verifique JDK 17 o superior compatible, Maven, Node.js/npm, PostgreSQL y Tomcat 10.1.
2. Cree/restaure la base y compruebe cuentas/tabla usuario desde pgAdmin.
3. Configure Backend/.env y Frontend/.env; verifique ambos dominios locales.
4. Compile el backend con Maven y despliegue el archivo WAR en webapps de Tomcat.
5. Ejecute npm install y npm run dev -- --port 5174 en Frontend.
6. Compruebe inicio de sesión y consulta autenticada.

```
cd Backend
mvn clean package
# Copiar target/VETERINARIAITQ.war a TOMCAT_HOME/webapps/
# Iniciar Tomcat después de cargar las variables de Backend/.env
cd ../Frontend
npm install
npm run dev -- --port 5174
```

URL de prueba: http://localhost:5174/acceso. API de salud básica: http://localhost:8080/VETERINARIAITQ/ (mensaje «VETERINARIAITQ API activa»).

## 6. Backend en Render con Docker

Se publicó el servicio controlpets-api a partir de la rama Desarrollo con Root Directory Backend, Docker y región Virginia (US East). La compilación final utilizó Maven + Tomcat 10.1, deshabilitando el puerto de apagado 8005 para evitar interferencias.

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml ./
COPY src ./src
RUN mvn -B -DskipTests clean package
FROM tomcat:10.1-jdk17-temurin
RUN rm -rf /usr/local/tomcat/webapps/*
RUN sed -i 's/port="8005"/port="-1"/' /usr/local/tomcat/conf/server.xml
COPY --from=build /app/target/VETERINARIAITQ.war /usr/local/tomcat/webapps/VETERINARIAITQ.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

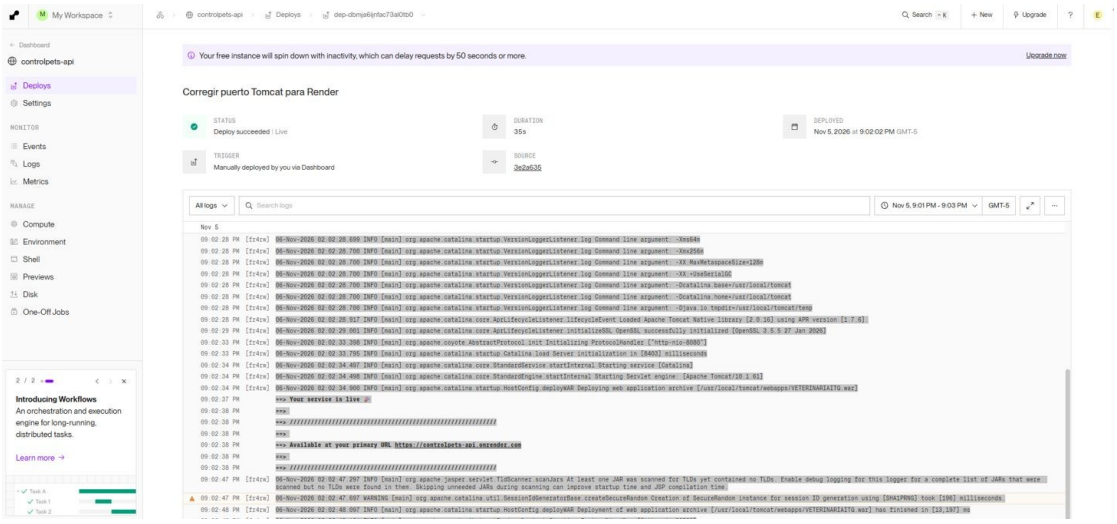



Figura 3. Registro real de la publicación Live del backend en Render.

En Render: Variables PORT/SERVER_PORT=8080, SERVER_CONTEXT_PATH=/VETERINARIAITQ, VET_DB_* para la base interna y JWT_SECRET. CORS_ORIGIN y FRONTEND_URL deben apuntar al dominio de Vercel. Nunca utilizar localhost para host de BD en Render.

## 7. Frontend en Vercel

El frontend se desplegó mediante Vercel CLI desde Frontend, sin vincular Git para auto-deploy. La URL canónica es https://controlpets.vercel.app; las actualizaciones del frontend se publican nuevamente con vercel --prod.

```
cd Frontend
npm install
vercel link
vercel env add VITE_API_URL production
# valor: https://controlpets-api.onrender.com/VETERINARIAITQ
vercel --prod
```

El archivo Frontend/vercel.json permite rutas del cliente al recargar /acceso o /app:

```
{"rewrites":[{"source":"/(.*)","destination":"/index.html"}]}
```

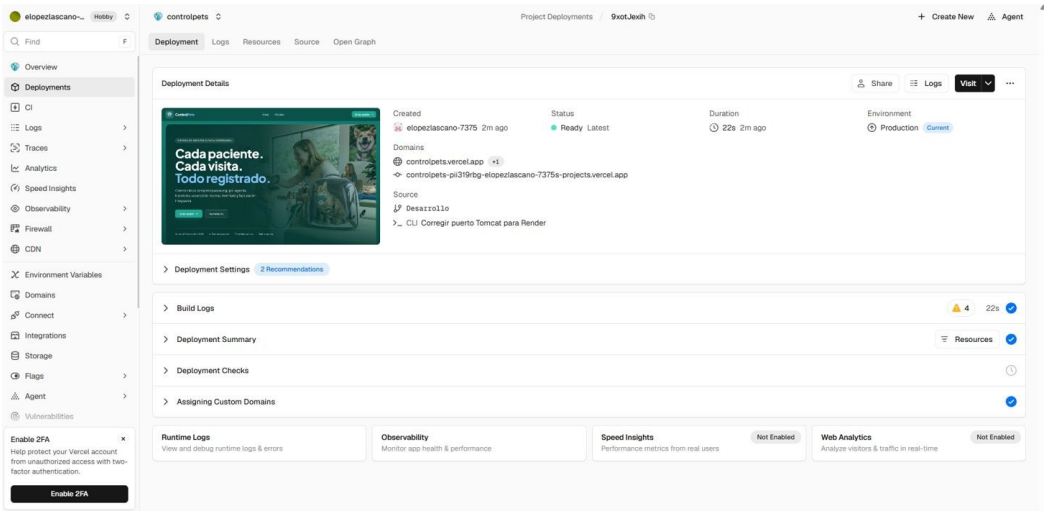



Figura 4. Despliegue de la interfaz de usuario en Vercel.

## 8. Seguridad, acceso y API REST

| Mecanismo | Implementación |
|---|---|
| Autenticación | POST /api/auth/login; credenciales verificadas con BCrypt y token JWT. |
| Autorización | Token Bearer y validación de roles/permisos en backend. |
| Multiempresa | Aislamiento por id_empresa en consultas y servicios según rol. |
| CORS | Origen autorizado por CORS_ORIGIN del backend. |
| Variables | JWT_SECRET, SMTP_PASSWORD y VET_DB_PASSWORD solo en secretos locales/servicios. |

```
POST https://controlpets-api.onrender.com/VETERINARIAITQ/api/auth/login
Content-Type: application/json
{ "usuario": "<USUARIO>", "clave": "<CONTRASENA>" }

GET /VETERINARIAITQ/api/empresas
Authorization: Bearer <TOKEN_JWT>
```

## 9. Actualizaciones y migraciones de base de datos

El código Git no modifica la base PostgreSQL de Render automáticamente. Mantenga scripts de migración versionados (por ejemplo migracion_bloqueo_mensualidades.sql); revíselos antes de aplicarlos en la instancia en línea.

7. Congelar cambios y exportar respaldo con pg_dump -Fc de la base cloud.
8. Revisar columnas, restricciones y dependencias del SQL; comprobar si la tabla existe.
9. Aplicar SQL con psql usando la URL externa de Render en conexión SSL.
10. Validar estructura y datos, luego desplegar Java con Manual Deploy y React con vercel --prod.
11. Ejecutar pruebas de regresión y conservar evidencia.

```
git switch Desarrollo
git pull --ff-only origin Desarrollo
# Revisar scripts SQL antes de ejecutarlos
psql "$URL_RENDER" -f Backend/src/main/resources/sql/migracion_x.sql
```

Al cierre del 5 de noviembre la instancia publicada contenía 24 tablas. En pruebas locales ya se observó una tabla mensualidad_empresa; por tanto no debe repetirse una migración sin inspeccionar la estructura. Los reportes/carnés de vacunas siguen pendientes.

## 10. Diagnóstico, continuidad y soporte

| Síntoma | Comprobación / acción |
|---|---|
| 404 al acceder desde React | Revisar VITE_API_URL y el rewrite de Vercel; compilar/publicar de nuevo. |
| 401 en login | Confirmar payload usuario/clave, cuentas y contexto de backend; no modificar hashes a ciegas. |
| 403 | Comprobar rol, permisos y estado de empresa. |
| Error de PostgreSQL | Validar host interno Render, nombre, usuario, contraseña y esquema. |
| Instancia suspendida | Render Free se reactiva al recibir una petición; para la presentación abrir anticipadamente. |
| Fallos SMTP | Revisar limitaciones del plan Free, credenciales y proveedor de correo. |

Antes de la evaluación: comprobar URLs públicas, autenticación desde navegador privado y dispositivo móvil, carga de registros y permisos. Respaldar DB y conservar el commit de despliegue. La instancia PostgreSQL gratuita mostraba fecha de vencimiento 5 de diciembre de 2026 en la captura proporcionada.

## 11. Secuencia de publicación reproducible

### 11.1. Base de datos

12. Crear PostgreSQL 18 en Render, misma región que el backend.
13. Identificar External Database URL para pg_restore desde el equipo; utilizar conexión cifrada.
14. Importar respaldo -Fc sobre una base vacía.
15. Comprobar total de tablas y existencia de cuentas de prueba sin mostrar contraseñas.

```
SELECT table_name FROM information_schema.tables
WHERE table_schema = 'public' ORDER BY table_name;
SELECT usuario, activo FROM public.usuario ORDER BY usuario;
```

### 11.2. Servicio Docker

16. Seleccionar el repositorio grupal, rama Desarrollo, lenguaje Docker y Root Directory Backend.
17. Comprobar que Dockerfile y pom.xml existan en la rama seleccionada.

18. Configurar PORT 8080, host interno de PostgreSQL, credenciales y JWT_SECRET como variables de entorno.
19. Iniciar Deploy y abrir Logs; buscar despliegue del WAR y estado Live.

### 11.3. SPA y CORS

20. Desde Frontend ejecutar vercel link y escoger el proyecto ControlPets.
21. Definir VITE_API_URL en Production con el dominio HTTPS de Render y contexto /VETERINARIAITQ.
22. Ejecutar vercel --prod y comprobar estado Ready.
23. Actualizar FRONTEND_URL y CORS_ORIGIN en Render a https://controlpets.vercel.app.
24. Validar login desde teléfono o navegación privada.

## 12. Plan de contingencia y restauración

| Incidente | Respuesta operativa |
|---|---|
| Pérdida de acceso Render | Verificar permisos de cuenta, estado del servicio y vigencia Free. |
| Despliegue nuevo rompe API | Volver al commit Docker/Java anterior desde Deploys y revisar Logs. |
| Cambio SQL falla | Detener siguiente despliegue, analizar transacción y restaurar backup si procede. |
| Frontend publicado con URL vieja | Actualizar VITE_API_URL en Vercel y ejecutar un despliegue nuevo. |
| Contraseña expuesta | Revocar credenciales en proveedor, actualizar variables y reiniciar servicio. |
| Expiración del plan gratuito | Exportar copia de PostgreSQL antes de la fecha de eliminación indicada por Render. |

## 13. Checklist de liberación y trazabilidad

| Control | Evidencia esperada |
|---|---|
| Versión | Commit SHA y rama desplegada anotados. |
| Compilación | Build Maven y Vite finalizados sin fallos. |
| Base de datos | Conexión Render probada, migración aplicada y conteo de tablas. |
| Seguridad | Variables privadas protegidas y CORS limitado al dominio de Vercel. |
| Funcional | Login, panel y al menos un flujo CRUD probado por rol. |
| Documentación | Manual técnico, usuario, QA y gestión actualizados en GitHub. |

## 14. Diferencias entre local y producción

| Local (equipo) | Cloud (Render/Vercel) |
|---|---|

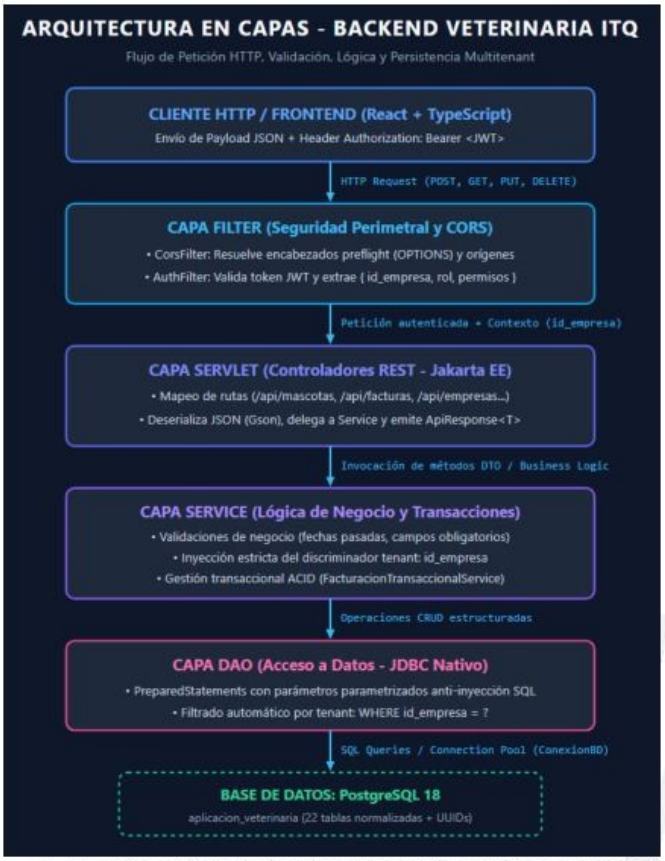
| | |
|---|---|
| PostgreSQL localhost:5432 / aplicacion_veterinaria | PostgreSQL Render / controlpets |
| Tomcat local en http://localhost:8080 | Tomcat Docker en HTTPS del Web Service |
| Vite en http://localhost:5174 | Vercel en https://controlpets.vercel.app |
| Archivo .env privado de cada integrante | Environment variables del servicio cloud |
| Cambios localmente inmediatos | Requiere nuevo deploy; SQL separado si cambia esquema |

## 15. Anexo visual de arquitectura y persistencia

Las siguientes imágenes corresponden a páginas de evidencia del manual técnico original. Se conservan por trazabilidad histórica; las cifras o rutas anteriores a esta revisión prevalecen únicamente como antecedente.

**Anexo T-07. Evidencia institucional histórica**

INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA




- **Paquete** config**:** Inicialización del pool de conexión y variables de entorno.
- **Paquete** filter**:** Interceptores de peticiones HTTP para control de acceso y seguridad perimetral.
- **Paquete** servlet**:** Endpoints mapeados que gestionan las operaciones doGet, doPost, doPut y doDelete.
- **Paquete** service**:** Coordinador de la lógica funcional, orquestación transaccional y validación de reglas de negocio.
- **Paquete** dao**:** Persistencia atómica mediante JDBC contra la base de datos aplicacion_veterinaria.

(+593) 96 356 1961 admisiones@itq.edu.ec Antonio de Ulloa N28-30 y Diego de Atienza (Esq). WWW.ITQ.EDU.EC

Extracto: Manuales_Tecnico_final.pdf original, página 7.

**Anexo T-12. Evidencia institucional histórica**

INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

8. Anexos

Esquema de Tablas

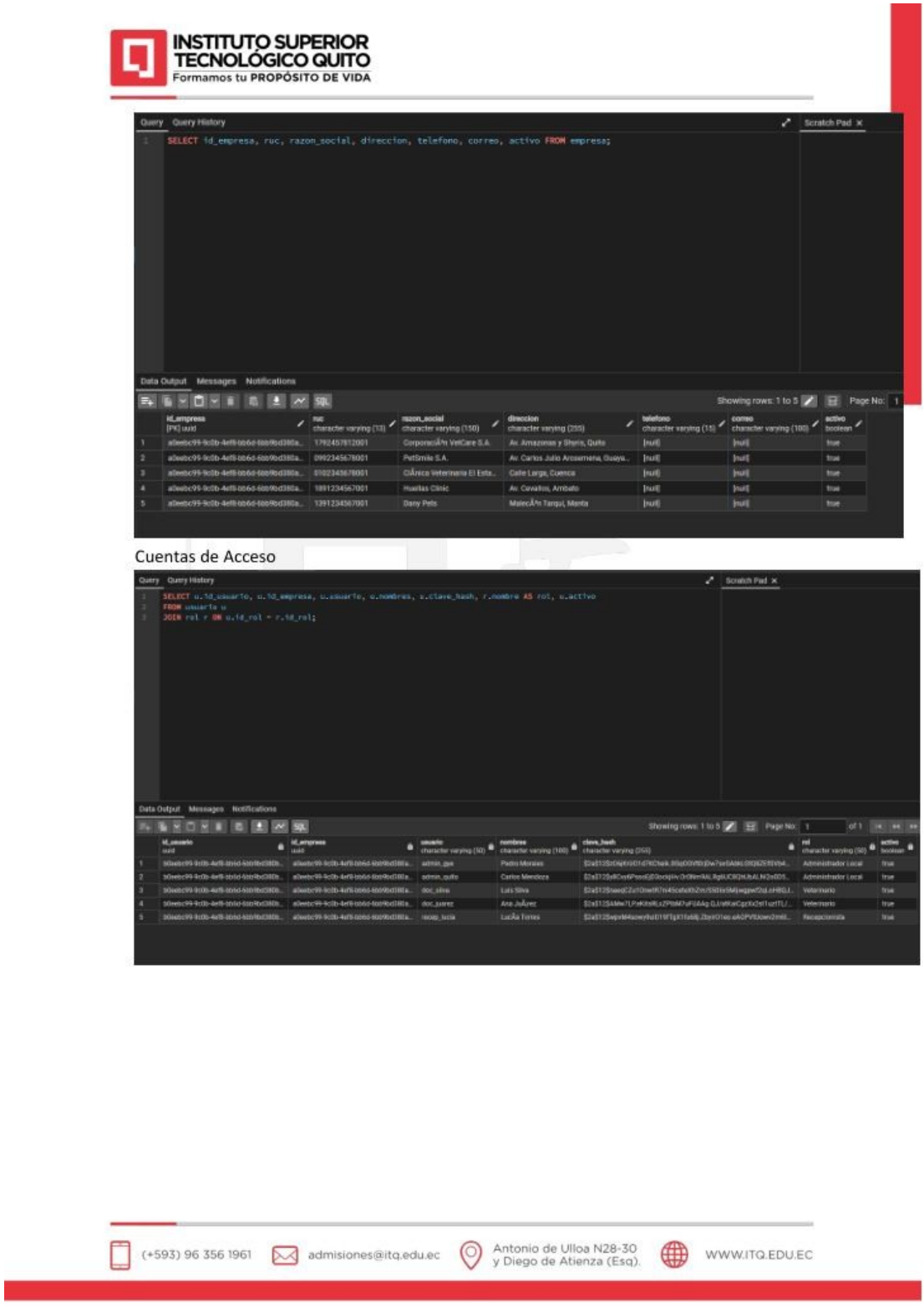
```
                List of tables
 Schema |          Name          | Type  |  Owner
--------+------------------------+-------+----------
 public | categoria              | table | postgres
 public | cita                   | table | postgres
 public | cliente                | table | postgres
 public | empresa                | table | postgres
 public | especie                | table | postgres
 public | factura                | table | postgres
 public | factura_detalle        | table | postgres
 public | historial_clinico      | table | postgres
 public | mascota                | table | postgres
 public | mascota_vacuna         | table | postgres
 public | movimiento_inventario  | table | postgres
 public | permiso                | table | postgres
 public | producto               | table | postgres
 public | raza                   | table | postgres
 public | receta                 | table | postgres
 public | receta_detalle         | table | postgres
 public | rol                    | table | postgres
 public | rol_permiso            | table | postgres
 public | sri_iva                | table | postgres
 public | usuario                | table | postgres
 public | vacuna                 | table | postgres
 public | veterinario            | table | postgres
(22 rows)
```

Tenencia (Clínicas)

(+593) 96 356 1961 admisiones@itq.edu.ec Antonio de Ulloa N28-30 y Diego de Atienza (Esq). WWW.ITQ.EDU.EC

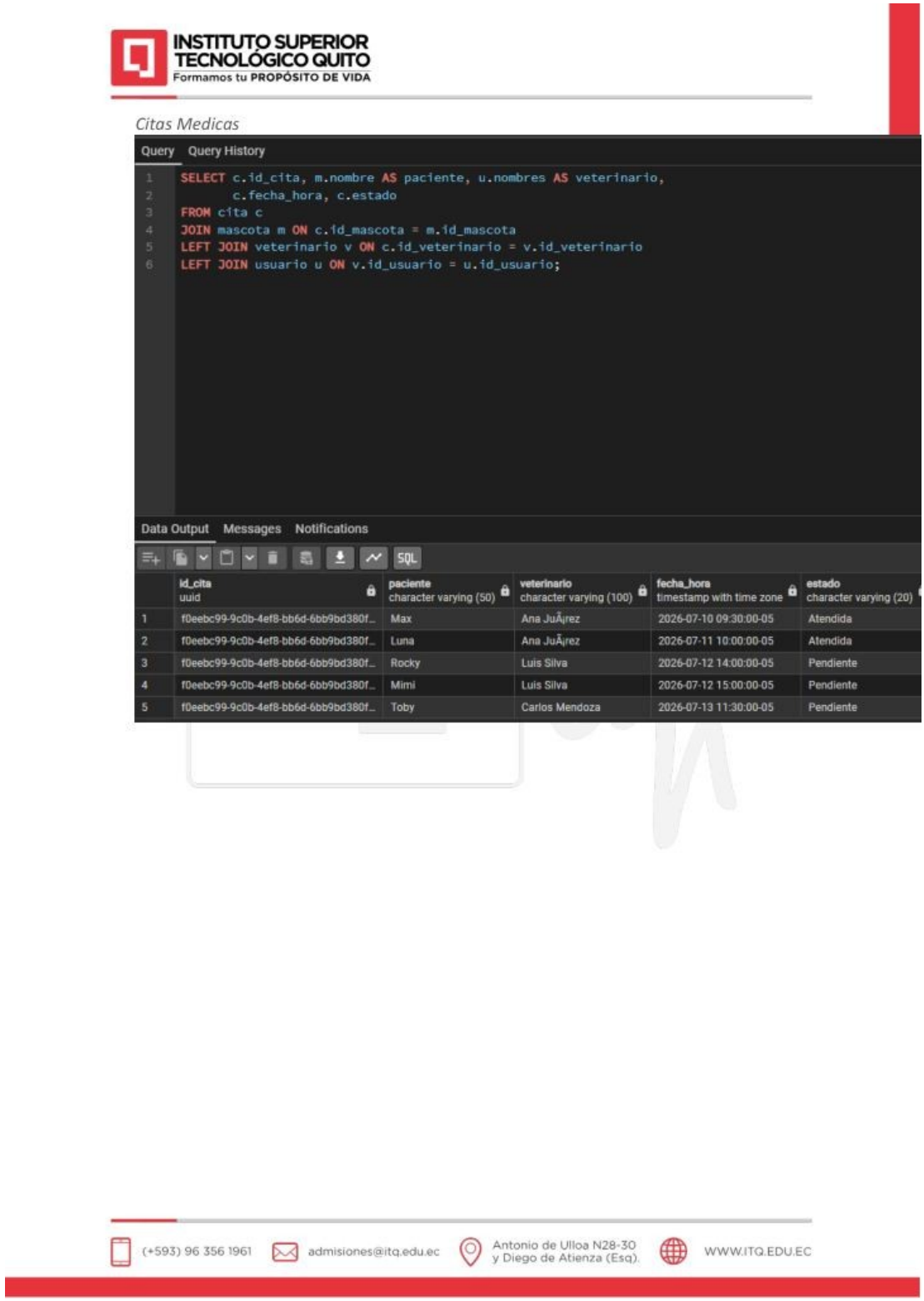
Extracto: Manuales_Tecnico_final.pdf original, página 12.

## Anexo T-13. Evidencia institucional histórica




Extracto: Manuales_Tecnico_final.pdf original, página 13.

**Anexo T-15. Evidencia institucional histórica**




**Extracto: Manuales_Tecnico_final.pdf original, página 15.**

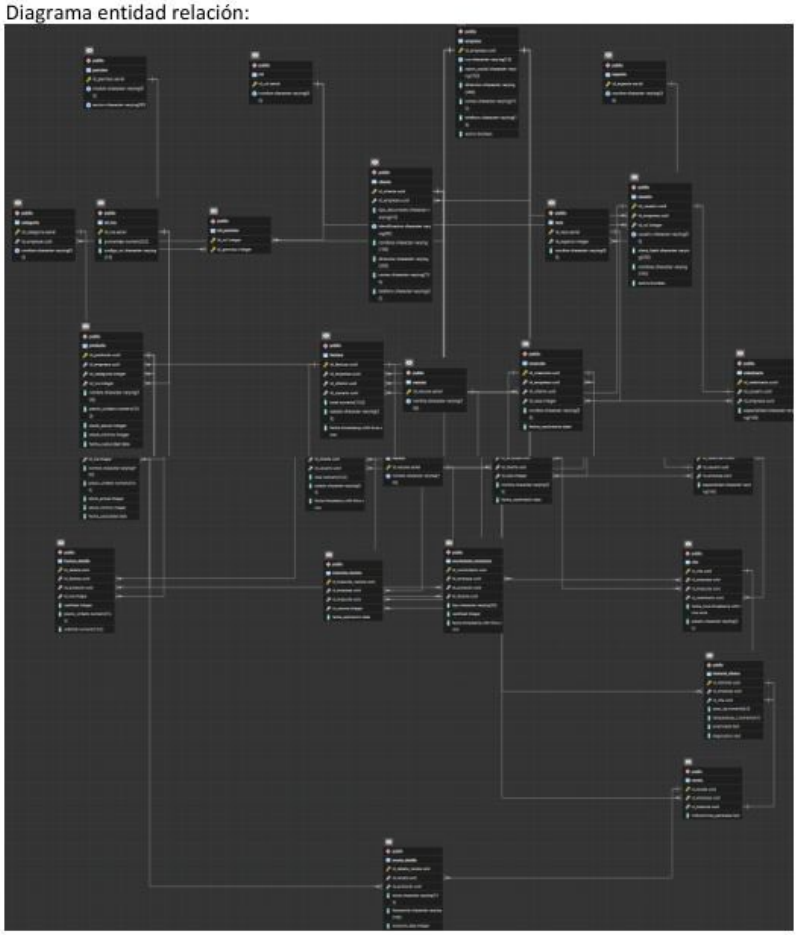
## Anexo T-17. Evidencia institucional histórica

INSTITUTO SUPERIOR TECNOLÓGICO QUITO
Formamos tu PROPÓSITO DE VIDA

*Facturación:*

| | id_factura uuid | total numeric (10,2) | estado character varying (20) | fecha timestamp with time zone | cantidad integer | precio_unitario numeric (10,2) | subtotal numeric (10,2) |
|---|---|---|---|---|---|---|---|
| 1 | 80eebc99-9c0b-4ef8-bb6d-6bb9bd3808... | 57.00 | Emitida | 2026-09-03 22:06:59.290307-... | 1 | 12.00 | 12.00 |
| 2 | 80eebc99-9c0b-4ef8-bb6d-6bb9bd3808... | 57.00 | Emitida | 2026-09-03 22:06:59.290307-... | 1 | 45.00 | 45.00 |
| 3 | 80eebc99-9c0b-4ef8-bb6d-6bb9bd3808... | 38.50 | Emitida | 2026-09-03 22:06:59.290307-... | 1 | 38.50 | 38.50 |
| 4 | 80eebc99-9c0b-4ef8-bb6d-6bb9bd3808... | 32.00 | Emitida | 2026-09-03 22:06:59.290307-... | 1 | 32.00 | 32.00 |
| 5 | 80eebc99-9c0b-4ef8-bb6d-6bb9bd3808... | 17.00 | Anulada | 2026-09-03 22:06:59.290307-... | 2 | 8.50 | 17.00 |

Diagrama entidad relación:



(+593) 96 356 1961 admisiones@itq.edu.ec Antonio de Ulloa N28-30 y Diego de Atienza (Esq). WWW.ITQ.EDU.EC

Extracto: Manuales_Tecnico_final.pdf original, página 17.